Relational Algebra & Relational Calculus

Formal Relational Query Languages

Two mathematical Query Languages form the basis for "real" languages (e.g. SQL), and for implementation:

- *Relational Algebra*: More operational, very useful for representing execution plans.
- *Relational Calculus*: Lets users describe what they want, rather than how to compute it. (Non-operational, *declarative*.)

- *Understanding Algebra & Calculus is key to*
- *understanding SQL, query processing!*

Relational Algebra

» Unary Relational Operations

» Relational Algebra Operations From Set Theory

» Binary Relational Operations

» Additional Relational Operations

» Examples of Queries in Relational Algebra

Relational Calculus

» Tuple Relational Calculus

» Domain Relational Calculus

Relational Algebra

Preliminaries

- Relational algebra is the basic set of operations for the relational model
- These operations enable a user to specify basic retrieval requests (or queries)
- The result of an operation is a new relation, which may have been formed from one or more input relations
    - » This property makes the algebra “closed” (all objects in relational algebra are relations)

Preliminaries

The algebra operations thus produce new relations

» These can be further manipulated using operations of the same algebra

A sequence of relational algebra operations forms a relational algebra expression

» The result of a relational algebra expression is also a relation that represents the result of a database query (or retrieval request)

Relational Algebra consists of several groups of operations

» Unary Relational Operations
- SELECT (symbol: (sigma))
- PROJECT (symbol: (pi))
- RENAME (symbol: (rho))

» Relational Algebra Operations From Set Theory
- UNION, INTERSECTION, DIFFERENCE (or MINUS, –)
- CARTESIAN PRODUCT (x)

» Binary Relational Operations
- JOIN (several variations of JOIN exist)
- DIVISION

» Additional Relational Operations
- OUTER JOINS
- AGGREGATE FUNCTIONS (These compute summary of information: for example, SUM, COUNT, AVG, MIN, MAX)

Unary Relational Operations: SELECT (1/3)

The SELECT operation (denoted by (sigma)) is used to select a subset of the tuples from a relation based on a selection condition.

» The selection condition acts as a filter

» Keeps only those tuples that satisfy the qualifying condition

» Tuples satisfying the condition are selected whereas the other tuples are discarded (filtered out)

Examples:

» Select the EMPLOYEE tuples whose department number is 4: DNO = 4 (EMPLOYEE)

» Select the employee tuples whose salary is greater than $30,000: SALARY > 30,000 (EMPLOYEE)

Unary Relational Operations: SELECT (2/3)

» In general, the select operation is denoted by <selection condition>(R) where

- the symbol (sigma) is used to denote the select operator
- the selection condition is a Boolean (conditional) expression specified on the attributes of relation R
- tuples that make the condition true are selected
  - appear in the result of the operation
- tuples that make the condition false are filtered out
  - discarded from the result of the operation

Unary Relational Operations: SELECT and PROJECT (2/3)

Example:
<selection condition> applied independently to each individual tuple t in R
If condition evaluates to TRUE, tuple selected
Boolean conditions AND, OR, and NOT
Unary
Applied to a single relation

Unary Relational Operations: SELECT (3/3)

SELECT Operation Properties

» The SELECT operation <selection condition>(R) produces a relation S that has the same schema (same attributes) as R

» SELECT is commutative:

 <condition1>(< condition2> (R)) = <condition2> (< condition1> (R))

» Because of commutativity property, a cascade (sequence) of SELECT operations may be applied in any order:

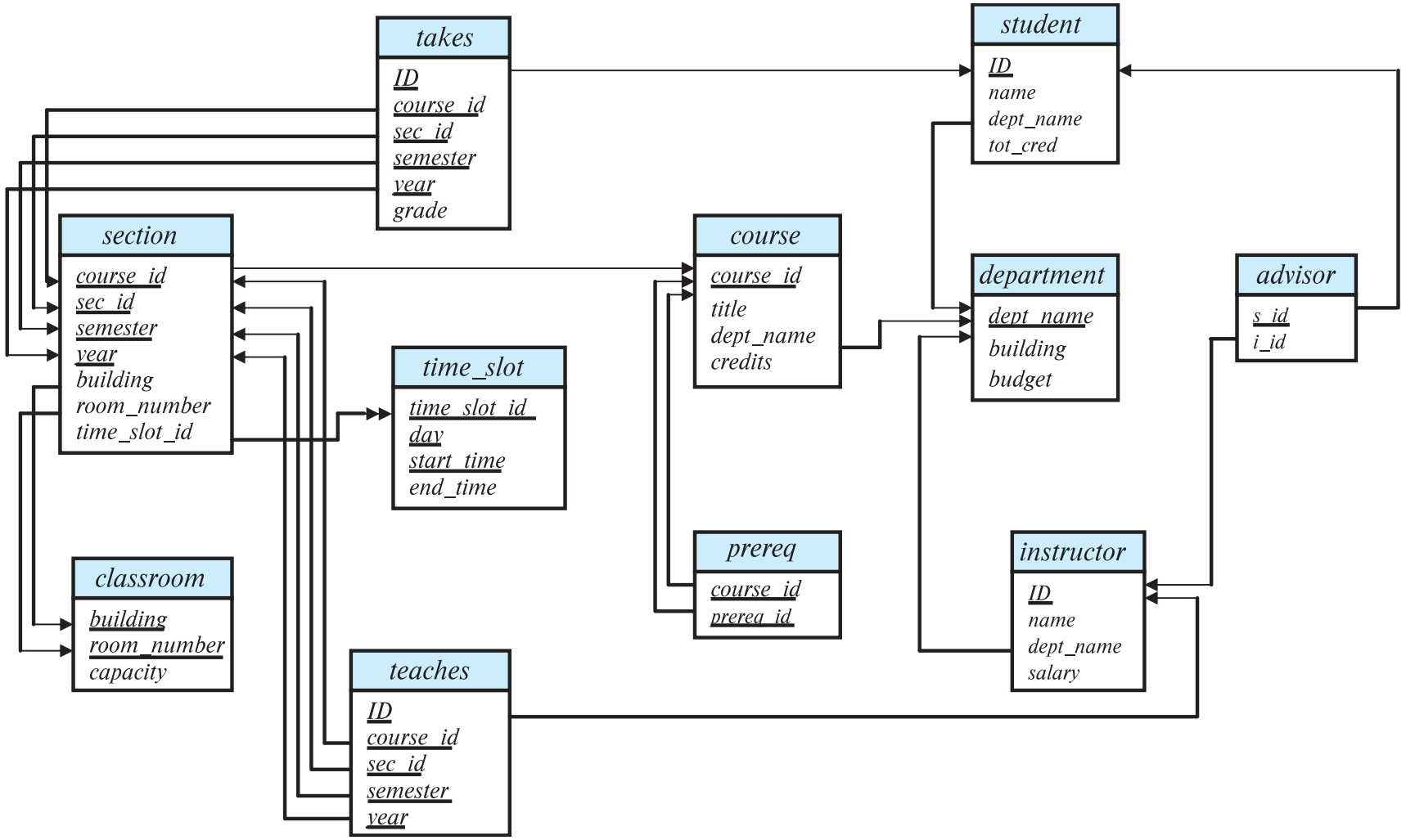
<cond1>(<cond2> (<cond3> (R)) = <cond2> (<cond3> (<cond1> (R)))

» A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:

<cond1>(< cond2> (<cond3>(R)) = <cond1> AND < cond2> AND < cond3>(R)))

» The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R

Schema Diagram for University Database



Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- *p* is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the "Physics" department.
  - Query

$$\sigma_{dept_name=\text{“Physics”}}(instructor)$$

  - Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation (Cont.)

- We allow comparisons using

  $=, \neq, >, \geq, <, \leq$

  in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:

  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Example: Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept_name=\text{“Physics”} \wedge salary > 90,000}(instructor)$$

- The select predicate may include comparisons between two attributes.
  - Example, find all departments whose name is the same as their building name:
  - $\sigma_{dept_name=building}(department)$

Project Operation

- A unary operation that returns its argument relation, with certain attributes left out.
- Notation:

$$\Pi_{A_1, A_2, A_3 \ldots A_k} (r)$$

  where A_1, A_2, …, A_k are attribute names and r is a relation name.
- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- ***Duplicate rows removed from result, since relations are sets***

Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*
- Query*:*

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

- Result:

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept_name = \text{“Physics”}} (instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

Cartesian-Product Operation

- The Cartesian-product operation (denoted by X) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and t*eaches* is written as:

  instructor X *teaches*

- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
  - *instructor.ID*
  - *teaches.ID*

The *instructor* X *teaches table*

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Cartesian Product Example

- **Relations *r*, *s*:**

A	*B*
α	1
β	2

r

C	*D*	*E*
α	10	*a*
β	10	*a*
β	20	*b*
γ	10	*b*

s

- ***r* x *s*:**

A	*B*	*C*	*D*	*E*
α	1	α	10	*a*
α	1	β	10	*a*
α	1	β	20	*b*
α	1	γ	10	*b*
β	2	α	10	*a*
β	2	β	10	*a*
β	2	β	20	*b*
β	2	γ	10	*b*

Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- For $r \cup s$ to be valid.
  1. *r, s* must have the *same* **arity** (same number of attributes)
  2. The attribute domains must be **compatible** (example: 2nd column of *r* deals with the same type of values as does the 2nd column of *s*)
- Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup$$
$$\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

Union Operation (Cont.)

- Result of:

$$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
  - r, s have the *same arity*
  - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\prod_{course_id} (\sigma_{semester=\text{“Fall”} \wedge year=2017} (section)) \cap \prod_{course_id} (\sigma_{semester=\text{“Spring”} \wedge year=2018} (section))$$

- Result

course_id
CS-101

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
  - r and s must have the same arity
  - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) - \Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-347
PHY-101

The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the "Physics" and Music department.

$$Physics \leftarrow \sigma_{dept_name=\text{“Physics”}}(instructor)$$
$$Music \leftarrow \sigma_{dept_name=\text{“Music”}}(instructor)$$
$$Physics \cup Music$$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.

The Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ, is provided for that purpose
- The expression:

$$\rho_x (E)$$

returns the result of expression *E* under the name *x*

- Another form of the rename operation:

$$\rho_{x(A1,A2, .. An)} (E)$$

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department with salary greater than 90,000
- Query 1

$$\sigma_{dept_name=\text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

- Query 2

$$\sigma_{dept_name=\text{“Physics”}}(\sigma_{salary > 90.000}(instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department
- Query 1

$$\sigma_{dept_name=\text{“Physics”}}(instructor \bowtie_{instructor.ID = teaches.ID} teaches)$$

- Query 2

$$(\sigma_{dept_name=\text{“Physics”}}(instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.

Derived Operators
Join and Division

Join Operation

- The Cartesian-Product

  instructor X *teaches*

  associates every tuple of instructor with every tuple of teaches.
  - Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of "*instructor* X *teaches* " that pertain to instructors and the courses that they taught, we write:

  $\sigma_{instructor.id = teaches.id}$ (*instructor* x *teaches*))

  - We get only those tuples of "*instructor* X *teaches*" that pertain to instructors and the courses that they taught.

Join Operation (Cont.)

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation (Cont.)

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations *r* (*R*) and *s* (*S*)
- Let "theta" be a predicate on attributes in the schema R "union" S. The join operation r $\bowtie_\theta$ s is defined as follows:

$$r \bowtie_\theta s = \sigma_\theta (r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches))$$

- Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches.$$

Binary Relational Operations: JOIN and DIVISION

- The **JOIN** Operation
  - Denoted by ⋈
  - Combine related tuples from two relations into single “longer” tuples
  - General join condition of the form <condition> **AND** <condition> **AND**...**AND** <condition>
  - Example:

$$\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{Mgr_ssn=Ssn}} \text{EMPLOYEE}$$
$$\text{RESULT} \leftarrow \pi_{\text{Dname, Lname, Fname}}(\text{DEPT_MGR})$$

JOIN

- **THETA** JOIN
  - Each <condition> of the form A_i θ B_j
  - A_i is an attribute of R
  - B_j is an attribute of S
  - A_i and B_j have the same domain
  - θ (theta) is one of the comparison operators:
    - {=, <, ≤, >, ≥, ≠}

Retrieve students enrolled in courses with more than 3 credits

SELECT student.name, course.title, course.credits FROM student, takes, course WHERE student.ID = takes.ID AND takes.course_id = course.course_id AND course.credits > 3;

Variations of JOIN: The EQUIJOIN and NATURAL JOIN

- **EQUIJOIN**
  - Only = comparison operator used
  - Always have one or more pairs of attributes that have identical values in every tuple

Retrieve student names and their enrolled courses using explicit join condition

SELECT student.name, course.title FROM student INNER JOIN takes ON student.ID = takes.ID INNER JOIN course ON takes.course_id = course.course_id;

NATURAL JOIN

Denoted by *

Removes second (superfluous) attribute in an EQUIJOIN condition

Retrieve student names along with the courses they are enrolled in

SELECT student.name, course.title FROM student NATURAL JOIN takes NATURAL JOIN course;

Variations of JOIN: The EQUIJOIN and NATURAL JOIN (cont'd.)

- **Inner/Outer joins**
  - Type of match and combine operation
  - Defined formally as a combination of CARTESIAN PRODUCT and SELECTION

Left Outer Join(A ⟕ B)

In the left outer join, operation allows keeping all tuple in the left relation. However, if there is no matching tuple is found in right relation, then the attributes of right relation in the join result are filled with null values.



Consider the following 2 Tables

A	
Num	Square
2	4
3	9
4	16

B	
Num	Cube
2	8
3	18
5	75

A ⋈ B		
Num	Square	Cube
2	4	8
3	9	18
4	16	–

Right Outer Join (A ⟖ B)

In the right outer join, operation allows keeping all tuple in the right relation. However, if there is no matching tuple is found in the left relation, then the attributes of the left relation in the join result are filled with null values.



A ⋈ B		
Num	Cube	Square
2	8	4
3	18	9
5	75	

Full Outer Join (A ⟗ B)

In a full outer join, all tuples from both relations are included in the result, irrespective of the matching condition.

A ⋈ B		
Num	Cube	Square
2	4	8
3	9	18
4	16	–
5	–	75

Division

- Not supported as a primitive operator, but useful for expressing queries like:

  *Find sailors who have reserved **all** boats*.

- Typical set-up: *A* has 2 fields *(x,y)* that are foreign key pointers, *B* has 1 matching field (*y*).

- Then *A/B* returns the set of *x*'s that match all *y* values in *B*.

- Example: *A* = Friend(x,y). *B* = set of 354 students. Then *A/B* returns the set of all x's that are friends with all 354 students.

Examples of Division A/B

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

A

pno
p2

B1

pno
p2
p4

B2

pno
p1
p2
p4

B3

sno
s1
s2
s3
s4

A/B1

sno
s1
s4

A/B2

sno
s1

A/B3

Find students who have taken all courses in the 'Computer Science' department

Step 1: Retrieve all course IDs for the "Computer Science" department.

$$CS_Courses \leftarrow \pi_{course_id}(\sigma_{dept_name='Computer Science'}(course))$$

Step 2: Retrieve students who have taken these courses.

$$Student_Courses \leftarrow \pi_{ID,course_id}(takes)$$

Step 3: Find students who have taken **all** courses in "Computer Science". This is done using **division** ($\div$).

$$Qualified_Students \leftarrow \pi_{ID}(Student_Courses \div CS_Courses)$$

Step 4: Retrieve student names.

$$Result \leftarrow \pi_{ID,name}(Qualified_Students \bowtie student)$$

Find instructors who have taught all available courses

Step 1: Retrieve all course IDs

$$All_Courses \leftarrow \pi_{course_id}(course)$$

Step 2: Retrieve instructor-course pairs

$$Instructor_Courses \leftarrow \pi_{ID,course_id}(teaches)$$

Step 3: Find instructors who have taught all courses

Using **division** ($\div$):

$$Qualified_Instructors \leftarrow \pi_{ID}(Instructor_Courses \div All_Courses)$$

Step 4: Retrieve instructor names

$$Result \leftarrow \pi_{ID,name}(Qualified_Instructors \bowtie instructor)$$

Example Instances

- "Sailors" and "Reserves" relations for our examples.
- We'll use positional or named field notation.
- Assume that names of fields in query results are inherited from names of fields in query input relations.

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Joins

- *Condition Join*: $R \bowtie_C S = \sigma_C (R \times S)$

(sid)	sname	rating	age	(sid)	bid	da
22	dustin	7	45.0	58	103	11
31	lubber	8	55.5	58	103	11

$S1 \bowtie_{S1.sid < R1.sid} R1$

- *Result schema* same as that of cross-product.
- Fewer tuples than cross-product, might be able to compute more efficiently
- Sometimes called a *theta-join*.

Joins

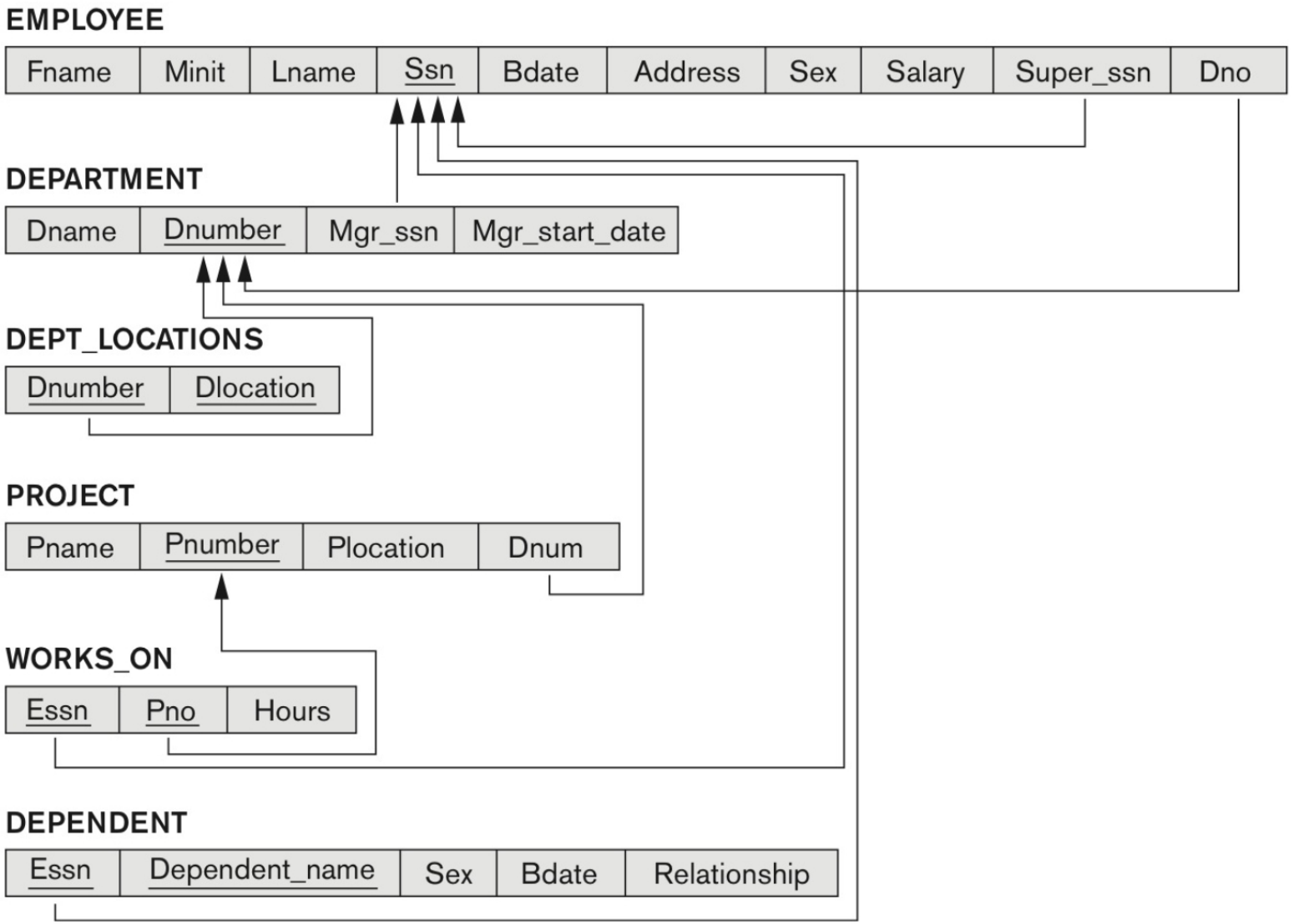
- *Equi-Join*: A special case of condition join where the condition *c* contains only ***equalities***.

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

$$S1 \bowtie_{R.sid = S.sid} R1$$

- *Result schema* similar to cross-product, but only one copy of fields for which equality is specified.
- *Natural Join*: Equijoin on *all* common fields.
  Without specified condition $A \bowtie B$ means the natural join of A and B.

Employee Database Schema



Examples of Queries in Relational Algebra :

Procedural Form

• Q1: Retrieve the name and Address of all Employees who work for the 'Research' department.

Research_dept ← σ Dname='Research' (Department)

Research_emps ← (RESEARCH_DEP DNumber= Dno Employee)

Result ← π Fname, Lname, Address (Research_emps)

As a single expression, this query becomes:

π Fname, Lname, Address (σ Dname= 'Research' (Department)

Dnumber=Dno Employee)

Examples of Queries in Relational Algebra :

Procedural Form

• Q2: Retrieve the names of Employees who have no dependents.

ALL_EMPS ← π SSN(Employee)

EMPS_WITH_DEPS(SSN) ← π Essn(DEPENDENT)

EMPS_WITHOUT_DEPS ← (ALL_EMPS - EMPS_WITH_DEPS)

RESULT ← π Lname, Fname (EMPS_WITHOUT_DEPS * Employee)

As a single expression, this query becomes:

π Lname, Fname((π Ssn (Employee) − ρ Ssn (πEssn (Dependent))) ∗ Employee)

1. Retrieve the names and addresses of all employees working in department number 5.

$$\pi_{Fname,Lname,Address}(\sigma_{Dno=5}(EMPLOYEE))$$

2. Find the names of employees who work on project number 10.

$$\pi_{Fname,Lname}(\sigma_{Pno=10}(EMPLOYEE \bowtie_{Ssn=Essn} WORKS_ON))$$

3. Retrieve the names of employees along with the department name they work in.

$$\pi_{Fname,Lname,Dname}(EMPLOYEE \bowtie_{Dno=Dnumber} DEPARTMENT)$$

4. List the names of employees who have dependents.

$$\pi_{Fname,Lname}(EMPLOYEE \bowtie_{Ssn=Essn} DEPENDENT)$$

5. Find the names of all employees who work on a project located in 'Houston'.

$$\pi_{Fname,Lname}(\sigma_{Plocation='Houston'}(EMPLOYEE \bowtie_{Ssn=Essn} WORKS_ON \bowtie_{Pno=Pnumber} PROJECT))$$

6. Retrieve the names of department managers.

$$\pi_{Fname,Lname}(EMPLOYEE \bowtie_{Ssn=Mgr_ssn} DEPARTMENT)$$

7. Find the employees who work on all projects in the company.

$$\pi_{Essn}(WORKS_ON) \div \pi_{Pnumber}(PROJECT)$$

8. Find the total number of employees in each department.

$$\pi_{Dno,COUNT(Ssn)}(EMPLOYEE)$$ (Aggregation - COUNT is not a core operator but can be assumed)

Relational Calculus

- Comes in two flavors: *Tuple relational calculus* (TRC) and *Domain relational calculus* (DRC).
- Calculus has *variables, constants, comparison ops*, *logical connectives* and *quantifiers*.
  - *TRC*: Variables range over (i.e., get bound to) *tuples*.
    - Like SQL.
  - *DRC*: Variables range over *domain elements* (= field values).
    - Like Query-By-Example (QBE)
  - Both TRC and DRC are simple subsets of first-order logic.
    - We'll focus on TRC here
- Expressions in the calculus are called *formulas*.
- Answer tuple is an assignment of constants to variables that make the formula evaluate to *true*.

Tuple Relational Calculus

- *Query* has the form: {*T* | *p*(*T*)}
  - ***p(T)*** denotes a formula in which tuple variable ***T*** appears.
- *Answer* is the set of all tuples *T* for which the *formula p*(*T*) evaluates to *true*.
- *Formula* is recursively defined:
  - start with simple *atomic formulas* (get tuples from relations or make comparisons of values)
  - build bigger and better formulas using the *logical connectives*.

TRC Formulas

- *An Atomic formula is one of the following:*

  $R \in Rel$

  $R.a$ *op* $S.b$

  $R.a$ *op constant*

  op is one of $<, >, =, \leq, \geq, \neq$

- A *formula* can be:
  - an atomic formula
  - $\neg p,\ p \wedge q,\ p \vee q$ where p and q are formulas
  - $\exists R(p(R))$ where variable *R* is a tuple variable
  - $\forall R(p(R))$ where variable *R* is a tuple variable

Common Notations Used in Relational Calculus

In relational calculus, several notations are used to form queries and express conditions logically. Here are some of the most common notations:

Tuple Variables: Used to represent rows in a relation (table). For example, a tuple variable t might represent a record in a “Students” table.

•**Predicate:** A logical condition that evaluates to true or false. It is used to filter records. For example, t.Age > 18 is a predicate that checks if a student’s age is above 18.

•**Logical Connectives:**

- **AND (∧):** Combines conditions that must both be true. **Example:**Age > 18 ∧ t.Grade = ‘A’.
- **OR (∨):** At least one of the conditions must be true. **Example:**Age < 18 ∨ t.Grade = ‘B’.
- **NOT (¬):** Negates a condition. **Example:** ¬(t.Age < 18).

- **Quantifiers:**
  - **Universal Quantifier (∀):** Means “for all.” It specifies that a condition must be true for all values. **Example:** ∀t (t.Age > 18) means all records have an age greater than 18.
  - **Existential Quantifier (∃):** Means “there exists.” It specifies that at least one value satisfies the condition. **Example:** ∃t (t.Grade = ‘A’) means there is at least one student with an “A” grade.
- **Equality and Comparison Operators:**
  - Common operators like =, <, >, ≤, and ≥ are used to compare attribute values.

Free and Bound Variables

- The use of quantifiers $\exists X$ and $\forall X$ in a formula is said to *bind* X in the formula.
  - A variable that is not bound is free.
- Let us revisit the definition of a query:
  - $\{T \mid p(T)\}$
- **There is an important restriction**
  - the variable T that appears to the left of `|' must be the *only* free variable in the formula $p(T)$.
  - in other words, all other tuple variables must be bound using a quantifier.

- *branch (branch_name, branch_city, assets)*
- *customer (customer_name, customer_street, customer_city)*
- *account (account_number, branch_name, balance)*
- *loan (loan_number, branch_name, amount)*
- *depositor (customer_name, account_number)*
- *borrower (customer_name, loan_number)*

- **Find the *loan_number*, *branch_name*, and *amount* for loans of over $1200**

$$\{t \mid t \in loan \wedge t[amount] > 1200\}$$

- **Find the loan number for each loan of an amount greater than $1200**

$$\{t \mid \exists s \in \text{loan}\ (t[loan_number] = s[loan_number] \wedge s[amount] > 1200)\}$$

Notice that a relation on schema [*loan_number*] is implicitly defined by the query

- Find the names of all customers having a loan, an account, or both at the bank

$$\{t \mid \exists s \in borrower\,(\,t[customer_name] = s[customer_name]\,)$$
$$\vee \exists u \in depositor\,(\,t[customer_name] = u[customer_name]\,)\}$$

- Find the names of all customers who have a loan and an account at the bank

$$\{t \mid \exists s \in borrower\,(\,t[customer_name] = s[customer_name]\,)$$
$$\wedge \exists u \in depositor\,(\,t[customer_name] = u[customer_name]\,)\}$$

- Find the names of all customers having a loan at the Perryridge branch

$$\{t \mid \exists s \in \textit{borrower}\,(t[\textit{customer_name}] = s[\textit{customer_name}] \wedge \exists u \in \textit{loan}\,(u[\textit{branch_name}] = \text{“Perryridge”} \wedge u[\textit{loan_number}] = s[\textit{loan_number}]))\}$$

- Find the names of all customers who have a loan at the Perryridge branch, but no account at any branch of the bank

$$\{t \mid \exists s \in \textit{borrower}\,(t[\textit{customer_name}] = s[\text{customer_name}] \wedge \exists u \in \textit{loan}\,(u[\textit{branch_name}] = \text{“Perryridge”} \wedge u[\textit{loan_number}] = s[\text{loan_}\textit{number}])) \wedge \text{not}\ \exists v \in \textit{depositor}\,(v[\textit{customer_name}] = t[\textit{customer_name}])\}$$

- Find the names of all customers having a loan from the Perryridge branch, and the cities in which they live

$$\{t \mid \exists s \in loan\,(s[branch_name] = \text{“Perryridge”} \wedge \exists u \in borrower\,(u[loan_number] = s[loan_number] \wedge t[customer_name] = u[customer_name]) \wedge \exists v \in customer\,(u[customer_name] = v[customer_name] \wedge t[customer_city] = v[customer_city]))\}$$

- **Find the names of all customers who have an account at all branches located in Brooklyn:**

$$\{t \mid \exists r \in customer\,(t[customer_name] = r[customer_name]) \wedge$$
$$(\forall u \in branch\,(u[branch_city] = \text{“Brooklyn”} \Rightarrow$$
$$\exists s \in depositor\,(t[customer_name] = s[customer_name]$$
$$\wedge \exists w \in account\,(w[account_number] = s[account_number]$$
$$\wedge\,(w[branch_name] = u[branch_name])))))\}$$

Safety of Expressions

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{ t \mid \neg\, t \in r \}$ results in an infinite relation if the domain of any attribute of relation r is infinite
- To guard against the problem, we restrict the set of allowable expressions to safe expressions.
- An expression $\{t \mid P(t)\}$ in the tuple relational calculus is *safe* if every component of t appears in one of the relations, tuples, or constants that appear in P
  - NOTE: this is more than just a syntax condition.
    - E.g. $\{ t \mid t[A] = 5 \vee \textbf{true} \}$ is not safe --- it defines an infinite set with attribute values that do not appear in any relation or tuples or constants in P.

sid	*sname*	*rating*	*age*
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 4.15 An Instance $S3$ of Sailors

sid	*bid*	*day*
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 4.16 An Instance $R2$ of Reserves

bid	*bname*	*color*
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Figure 4.17 An Instance $B1$ of Boats

Find the names and ages of sailors with a rating above 7.

$$\{P \mid \exists S \in Sailors(S.rating > 7 \wedge P.name = S.sname \wedge P.age = S.age)\}$$

This query illustrates a useful convention: P is considered to be a tuple variable with exactly two fields, which are called name and age, because these are the only fields of P that are mentioned and P does not range over any of the relations in the query; that is, there is no subformula of the form P ∈ Relname. The result of this query is a relation with two fields, name and age. The atomic formulas P.name = S.sname and P.age = S.age give values to the fields of an answer tuple P . On instances B1, R2, and S3, the answer is the set of tuples 〈Lubber, 55.5〉, 〈Andy, 25.5〉, 〈Rusty, 35.0〉, 〈Zorba, 16.0〉, and 〈Horatio, 35.0〉

Find the sailor name, boat id, and reservation date for each reservation.

$$\{P \mid \exists R \in \text{Reserves}\ \exists S \in \text{Sailors}\ (R.sid = S.sid \wedge P.bid = R.bid \wedge P.day = R.day \wedge P.sname = S.sname)\}$$

For each Reserves tuple, we look for a tuple in Sailors with the same sid. Given a pair of such tuples, we construct an answer tuple P with fields sname, bid, and day by copying the corresponding fields from these two tuples. This query illustrates how we can combine values from different relations in each answer tuple

Find the names of sailors who have reserved boat 103.

$$\{P \mid \exists S \in Sailors\ \exists R \in Reserves(R.sid = S.sid \wedge R.bid = 103 \wedge P.sname = S.sname)\}$$

This query can be read as follows: “Retrieve all sailor tuples for which there exists a tuple in Reserves, having the same value in the sid field, and with bid = 103.” That is, for each sailor tuple, we look for a tuple in Reserves that shows that this sailor has reserved boat 103. The answer tuple P contains just one field, sname.

Find the names of sailors who have reserved a red boat.

$\{P \mid \exists S \in Sailors\ \exists R \in Reserves(R.sid = S.sid \wedge P.sname = S.sname \wedge \exists B \in Boats(B.bid = R.bid \wedge B.color = 'red'))\}$

This query can be read as follows: “Retrieve all sailor tuples S for which there exist tuples R in Reserves and B in Boats such that S.sid = R.sid, R.bid = B.bid, and B.color =’red’.” Another way to write this query, which corresponds more closely to this reading, is as follows:

$\{P \mid \exists S \in Sailors\ \exists R \in Reserves\ \exists B \in Boats\ (R.sid = S.sid \wedge B.bid = R.bid \wedge B.color = 'red' \wedge P.sname = S.sname)\}$

Domain Relational Calculus

- **Query** has the form:

$$\{\langle x1, x2, \ldots, xn \rangle \mid p(\langle x1, x2, \ldots, xn \rangle)\}$$

  - **Answer** includes all tuples $\langle x1, x2, \ldots, xn \rangle$ that make the *formula* $p(\langle x1, x2, \ldots, xn \rangle)$ be *true*.

  - ***Formula*** is recursively defined, starting with simple ***atomic formulas*** (getting tuples from relations or making comparisons of values), and building bigger and better formulas using the ***logical connectives***.

Domain Relational Formulae

- ***Atomic formula:***
  - $\langle x1, x2, ..., xn \rangle \in Rname$, or X op Y, or X op constant
  - op is one of $<, >, =, \leq, \geq, \neq$
- ***Formula:***
  - an atomic formula, or
  - $\neg p, p \wedge q, p \vee q$, where p and q are formulas, or
  - $\exists X(p(X))$, where variable X is *free* in p(X), or
  - $\forall X(p(X))$, where variable X is *free* in p(X)
- The use of quantifiers $\exists X$ and $\forall X$ is said to *bind* X.
  - A variable that is not bound is free.

Free and Bound Variables

- The use of **quantifiers** $\exists X$ and $\forall X$ in a formula is said to ***bind*** X.
  - A variable that is **not bound** is **free**.
- Let us revisit the definition of a **query**:

$$\{\langle x1, x2, \ldots, xn \rangle \mid p(\langle x1, x2, \ldots, xn \rangle)\}$$

- There is an important restriction: the variables **x1, …, xn** that appear to the left of `|' must be the ***only*** free variables in the formula p(…).

- *branch (branch_name, branch_city, assets)*
- *customer (customer_name, customer_street, customer_city)*
- *account (account_number, branch_name, balance)*
- *loan (loan_number, branch_name, amount)*
- *depositor (customer_name, account_number)*
- *borrower (customer_name, loan_number)*

- Find the *loan_number, branch_name,* and *amount* for loans of over $1200

$$\{< l, b, a > \mid < l, b, a > \in loan \wedge a > 1200\}$$

- **Find the names of all customers who have a loan of over $1200**

$$\{< c > \mid \exists\, l, b, a\, (< c, l > \in borrower \wedge < l, b, a > \in loan \wedge a > 1200)\}$$

- **Find the names of all customers who have a loan from the Perryridge branch and the loan amount:**
  - $\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge \exists b\, (< l, b, a > \in loan \wedge b = \text{“Perryridge”}))\}$
  - $\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge < l, \text{"Perryridge"}, a > \in loan)\}$

- **Find the names of all customers having a loan, an account, or both at the Perryridge branch:**

$$\{< c > \mid \exists\, l\, (< c, l > \in \textit{borrower} \wedge \exists\, b,a\, (< l, b, a > \in \textit{loan} \wedge b = \text{“Perryridge”})) \vee \exists\, a\, (< c, a > \in \textit{depositor} \wedge \exists\, b,n\, (< a, b, n > \in \textit{account} \wedge b = \text{“Perryridge”}))\}$$

  - **Find the names of all customers who have an account at all branches located in Brooklyn:**

$$\{< c > \mid \exists\, s,n\, (< c, s, n > \in \text{customer}) \wedge \forall\, x,y,z\, (< x, y, z > \in \textit{branch} \wedge y = \text{“Brooklyn”}) \Rightarrow \exists\, a,b\, (< x, y, z > \in \textit{account} \wedge < c,a > \in \textit{depositor})\}$$

Safety of Expressions

The expression:

$$\{ < x_1, x_2, \ldots, x_n > \mid P(x_1, x_2, \ldots, x_n) \}$$

is safe if all of the following hold:

1. All values that appear in tuples of the expression are values from *dom* (P) (that is, the values appear either in P or in a tuple of a relation mentioned in P).
2. For every "there exists" subformula of the form $\exists x\ (P_1(x))$, the subformula is true if and only if there is a value of x in *dom* (P_1) such that $P_1(x)$ is true.
3. For every "for all" subformula of the form $\forall_x\ (P_1\ (x))$, the subformula is true if and only if $P_1(x)$ is true for all values x from *dom* (P_1).

Examples of DRC Queries

Find all sailors with a rating above 7.

$\{\langle I, N, T, A\rangle \mid \langle I, N, T, A\rangle \in \text{Sailors} \wedge T > 7\}$

This differs from the TRC version in giving each attribute a (variable) name. The condition $\langle I, N, T, A\rangle \in \text{Sailors}$ ensures that the domain variables I, N , T , and A are restricted to be fields of the same tuple. In comparison with the TRC query, we can say T > 7 instead of S.rating > 7, but we must specify the tuple $\langle I, N, T, A\rangle$ in the result, rather than just S.

Find the names of sailors who have reserved boat 103.

$\{\langle N \rangle \mid \exists I, T, A(\langle I, N, T, A \rangle \in Sailors \wedge \exists Ir, Br, D(\langle Ir, Br, D \rangle \in Reserves \wedge Ir = I \wedge Br = 103))\}$

Notice that only the sname field is retained in the answer and that only N is a free variable. We use the notation $\exists Ir, Br, D(\ldots)$ as a shorthand for $\exists Ir(\exists Br(\exists D(\ldots)))$. Very often, all the quantified variables appear in a single relation, as in this example. An even more compact notation in this case is $\exists \langle Ir, Br, D \rangle \in Reserves$. With this notation, which we will use henceforth, the above query would be as follows:

$\{\langle N \rangle \mid \exists I, T, A(\langle I, N, T, A \rangle \in Sailors \wedge \exists \langle Ir, Br, D \rangle \in Reserves(Ir = I \wedge Br = 103))\}$

The comparison with the corresponding TRC formula should now be straightforward. This query can also be written as follows; notice the repetition of variable I and the use of the constant 103:

{〈N〉 | ∃I, T, A(〈I, N, T, A〉 ∈ Sailors ∧∃D(〈I, 103, D〉 ∈ Reserves))}

Find the names of sailors who have reserved a red boat.
{〈N〉 | ∃I, T, A(〈I, N, T, A〉 ∈ Sailors ∧∃〈I, Br, D〉 ∈ Reserves ∧ ∃〈Br, BN,'red'〉 ∈ Boats)}

Find the names of sailors who have reserved at least two boats.
{〈N〉 | ∃I, T, A(〈I, N, T, A〉 ∈ Sailors ∧ ∃Br1, Br2, D1, D2(〈I, Br1, D1〉 ∈ Reserves ∧ 〈I, Br2, D2〉 ∈ Reserves ∧ Br1 # = Br2))}

Notice how the repeated use of variable I ensures that the same sailor has reserved both the boats in question.

Find sailors who have reserved all red boats.

$\{ \langle I, N, T, A \rangle \mid \langle I, N, T, A \rangle \in Sailors \wedge \forall \langle B, BN, C \rangle \in Boats\ (C = \text{'red'} \Rightarrow \exists \langle Ir, Br, D \rangle \in Reserves(I = Ir \wedge Br = B))\}$

Here, we find all sailors such that for every red boat there is a tuple in Reserves that shows the sailor has reserved it.

Comparison Between TRC and DRC

Aspect	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variables Used	Uses tuple variables representing entire rows.	Uses domain variables representing individual attribute values.
Expression Structure	{ t | P(t) } where t is a tuple and `P(t)` is a predicate.	{<x1, x2, …, xn> | P(x1, x2, …, xn)}` where variables represent attributes.
Focus	Queries entire tuples (rows).	Queries specific attributes (domains) of tuples.
Result Format	Returns tuples that satisfy the condition.	Returns attribute values that meet the criteria.
Ease of Use	More natural for querying full rows and complex relationships.	Better suited for simple queries involving specific fields.
Example Query Style	{ t | t ? Students ? t.Grade = ‘A’ }	{<Name> | ? ID, Age, Grade (Students(ID, Name, Age, Grade) ? Age = 20)}
Common Use Case	Often used for retrieving entire records that match a certain criteria.	Used to fetch specific columns based on certain conditions.

Advantages of Relational Calculus

- **Simplicity:** Allows users to specify “what” data to retrieve without knowing “how” the query will be executed.
- **Flexibility:** Can represent a wide range of queries using logical conditions.
- **Theoretical Significance:** Serves as a foundation for understanding the principles of database management and query languages.
- **Declarative Nature:** Makes it easier to express complex queries compared to procedural approaches.
- **Logical Representation:** Provides a formal way to describe database queries mathematically.

Limitations of Relational Calculus in DBMS

- **Lack of Procedural Information:** Does not provide any indication of how the query will be executed, which can be inefficient for large datasets.
- **Complexity in Expression:** Writing queries with multiple conditions can become complicated and hard to understand.
- **Limited Practical Use**: Not directly used as a practical querying method in most real-world applications; more relevant for theoretical purposes.
- **Performance Concerns:** Since it is purely declarative, performance optimization may require additional steps.
- **Difficult Error Handling**: Diagnosing errors in relational calculus queries can be challenging due to the abstract nature of the expressions.

Thank You